

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и технологий
Высшая школа программной инженерии

Курсовая работа
по дисциплине «Объектно-ориентированное программирование»

Тема: «Система визуализации инерциальной навигации на основе полётного контроллера Pixhawk»

Выполнил
Студент гр. 3331506/30401

Томилов А. С.

Преподаватель

Ананьевский М. С.

«___» _____ 2026 г.

Санкт-Петербург
2026 г.

Содержание

Введение

Инерциальная навигация — один из фундаментальных методов определения положения и ориентации объекта в пространстве без использования внешних радионавигационных сигналов [?]. В основе метода лежат данные инерциального измерительного блока (IMU — *Inertial Measurement Unit*), включающего трёхосевые акселерометр и гироскоп. Интегрируя угловые скорости и линейные ускорения во времени, можно восстановить полную навигационную картину: ориентацию, скорость и положение объекта [?].

Полётный контроллер Pixhawk является широко распространённым стандартом в области автономных летательных аппаратов [?]. Он оснащён высококачественным МЭМС-IMU и поддерживает протокол MAVLink [?] — открытый телеметрический стандарт, позволяющий в реальном времени получать сырые данные датчиков на внешний компьютер [?].

Целью данной работы является разработка программной системы, которая осуществляет чтение сырых IMU-данных (сообщения RAW_IMU протокола MAVLink) с полётного контроллера Pixhawk и выполняет их обработку в реальном времени: вычисление ориентации объекта через кватернионное интегрирование гироскопа [?], а также оценку скорости и позиции посредством двойного интегрирования ускорения [?]. Результаты визуализируются в интерактивном 3D-окне с использованием OpenGL 3.3 и библиотеки GLFW [?].

Задачи работы:

1. реализовать автономный модуль первичного считывания данных по протоколу MAVLink через последовательный порт с записью в CSV;
2. разработать многопоточную архитектуру с потокобезопасным разделяемым буфером;

3. реализовать алгоритм калибровки и кватернионного интегрирования гироскопических данных;
4. реализовать алгоритм двойного интегрирования акселерометра для оценки позиции;
5. реализовать механизм ZUPT (*Zero Velocity Update*) для частичной компенсации дрейфа;
6. создать OpenGL-визуализацию с 3D-моделью полётного контроллера, траекторией движения и HUD-панелью графиков датчиков;
7. провести анализ точности полученных результатов.

1 Первичное считывание данных с Pixhawk

1.1 Протокол MAVLink и сообщение RAW_IMU

MAVLink (*Micro Air Vehicle Link*) — лёгкий бинарный протокол сериализации для беспилотных систем, разработанный Лоренцем Майером в 2009 году [?]. Протокол широко используется в экосистемах PX4 и ArduPilot [?]. Для получения сырых показаний датчиков применяется сообщение RAW_IMU (идентификатор 27 в диалекте `common`), содержащее девять целочисленных полей в единицах внутренней шкалы датчика (LSB — *Least Significant Bit*):

Таблица 1 – Поля сообщения MAVLink RAW_IMU

Поле	Датчик	Описание
хасс, уасс, засс	Акселерометр	Ускорение по осям X, Y, Z (LSB)
хgyro, уgyro, зgyro	Гироскоп	Угловая скорость по осям (LSB)
хmag, уmag, зmag	Магнитометр	Магнитное поле по осям (LSB)

1.2 Модуль первичного считывания

На первом этапе работы был написан автономный модуль `read_raw_data_from_pixhawk.cpp`, позволивший проверить корректность соединения с Pixhawk и собрать сырые данные для анализа и подбора калибровочных коэффициентов. Взаимодействие реализовано через последовательный порт `/dev/ttyACM0` на скорости 115200 бод. Инициализация порта выполнена через POSIX-интерфейс `termios` [?]: 8 бит данных, без паритета, один стоп-бит, без аппаратного управления потоком, таймаут 1 с.

```
1 int setup_serial_port(const char* port, int baudrate) {  
2     int fd = open(port, O_RDWR | O_NOCTTY);
```

```

3   struct termios tty;
4   memset(&tty, 0, sizeof(tty));
5   tcgetattr(fd, &tty);
6   cfsetospeed(&tty, baudrate);
7   cfsetispeed(&tty, baudrate);
8   tty.c_cflag |= (CLOCAL | CREAD);
9   tty.c_cflag &= ~CSIZE;
10  tty.c_cflag |= CS8;
11  tty.c_cflag &= ~(PARENB | CSTOPB | CRTSCTS);
12  tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
13  tty.c_iflag &= ~(IXON | IXOFF | IXANY);
14  tty.c_oflag &= ~OPOST;
15  tty.c_cc[VMIN] = 0;
16  tty.c_cc[VTIME] = 10;
17  tcsetattr(fd, TCSANOW, &tty);
18  return fd;
19 }

```

Листинг 1: Настройка последовательного порта через `termios`

MAVLink-парсер реализован в виде конечного автомата: `mavlink_parse_char()` возвращает ненулевое значение только при получении полного валидного пакета с проверкой CRC-16 [?]. Каждый принятый пакет `RAW_IMU` записывается в CSV-файл с меткой времени с точностью до микросекунды (`gettimeofday()`); принудительный сброс буфера выполняется каждые 50 пакетов.

1.3 Результаты первичного считывания

Сообщения `RAW_IMU` стабильно поступали с частотой порядка 100 Гц, что соответствует штатной конфигурации IMU на Pixhawk [?]. Сырые данные акселерометра в покое показали устойчивое значение по оси Z (проекция силы тяжести), а гироскоп — дрейф нуля, подлежащий компенсации на следующем этапе.

2 Архитектура системы визуализации

2.1 Общая структура

На основе опыта, полученного при реализации модуля чтения, была разработана полноценная система **Pixhawk IMU Visualizer** — многопоточное приложение на C++14. Два параллельных потока взаимодействуют через потокобезопасный разделяемый буфер (SharedBuffer):

Главный поток (`pixhawk_imu_reader.cpp`) устанавливает соединение с Pixhawk по UART, непрерывно парсит входящий байтовый поток и при получении пакета RAW_IMU помещает данные в очередь, а также дублирует их в CSV-файл.

Поток визуализации (`imu_visualizer.cpp`) извлекает пакеты из очереди, запускает алгоритм инерциальной навигации и обновляет 3D-сцену на каждом кадре.

2.2 Структура файлов проекта

Таблица 2 – Файлы проекта

Файл	Назначение
imu_shared.h	Общие структуры данных: RawIMU, NavState, SharedBuffer, константы калибровки
imu_visualizer.h	Объявление функции run_visualizer()
imu_visualizer.cpp	Навигационный алгоритм, GLSL-шейдеры, геометрия модели и OpenGL-рендер
pixhawk_imu_reader.cpp	MAVLink-ридер, запуск потока визуализации
read_raw_data_from_pixhawk.cpp	Автономный модуль первичного считывания
CMakeLists.txt	Система сборки CMake
c_library_v2/	Заголовочная библиотека MAVLink C [?]

2.3 Разделяемые структуры данных (imu_shared.h)

Вся общая информация инкапсулирована в трёх структурах. Структура RawIMU хранит одно необработанное измерение:

```
1 struct RawIMU {
2     double timestamp;
3     int16_t xacc, yacc, zacc;    // акселерометр (LSB)
4     int16_t xgyro, ygyro, zgyro; // гироскоп (LSB)
5     int16_t xmag, ymag, zmag;    // магнитометр (LSB)
6 };
7
8 struct NavState {
9     double qw=1, qx=0, qy=0, qz=0; // кватернион ориентации
10    double wx=0, wy=0, wz=0;        // угловые скорости радс(/)
11    double vx=0, vy=0, vz=0;        // скорость мс(/)
12    double px=0, py=0, pz=0;        // позиция м()
```



```

13     double roll=0, pitch=0, yaw=0; // углы Эйлера градусы()
14     double ax=0, ay=0, az=0;      // ускорение мс(/^2)
15 };

```

Листинг 2: Структуры RawIMU и NavState

Структура `SharedBuffer` объединяет всё межпоточное взаимодействие. Каждое поле защищено отдельным мьютексом, что позволяет потокам работать параллельно без глобальной блокировки. Флаг `running` объявлен как `std::atomic<bool>`:

```

1 struct SharedBuffer {
2     std::mutex          mtx;
3     std::deque<RawIMU>  queue;          // очередь пакетов макс(. 500)
4     static const int    MAX_QUEUE = 500;
5     std::atomic<bool>   running{true};
6     std::mutex          nav_mtx;
7     NavState            nav;
8     std::mutex          trail_mtx;
9     std::deque<std::array<float,3>> trail; // траектория макс(. 2000)
10    static const int     MAX_TRAIL = 2000;
11
12    void push(const RawIMU& r) {
13        std::lock_guard<std::mutex> lk(mtx);
14        if ((int)queue.size() < MAX_QUEUE) queue.push_back(r);
15    }
16    bool pop(RawIMU& out) {
17        std::lock_guard<std::mutex> lk(mtx);
18        if (queue.empty()) return false;
19        out = queue.front(); queue.pop_front(); return true;
20    }
21 };

```

Листинг 3: Потокобезопасный разделяемый буфер

3 Зависимости и система сборки

3.1 Используемые библиотеки

Таблица 3 – Внешние зависимости проекта

Библиотека	Версия	Роль в проекте
MAVLink <code>c_library_v2</code> [?]	2.0	Парсинг телеметрии, декодирование RAW_IMU
GLFW3 [?]	≥ 3.3	Создание окна OpenGL, ввод мыши и клавиатуры
GLEW	≥ 2.1	Загрузка расширений OpenGL 3.3
GLM	$\geq 0.9.9$	Матричная и кватернионная математика
pthread	POSIX	Межпоточная синхронизация (<code>std::thread</code> , <code>std::mutex</code>)

Библиотека MAVLink является заголовочной — она полностью состоит из `.h`-файлов и не требует компиляции [?]. Остальные зависимости устанавливаются из репозитория пакетов Ubuntu.

3.2 Конфигурация CMake

Сборочная система использует CMake версии не ниже 3.10 и настроена на стандарт C++14. Зависимости ищутся через `find_package` и `PkgConfig`; проект компилируется с флагами `-O2 -Wall -Wextra`. CMake ищет GLFW тремя способами последовательно: через `find_package`, через `PkgConfig` и, наконец, вручную через `find_path` / `find_library` — это обеспечивает совместимость с различными дистрибутивами Ubuntu.

4 Математика инерциальной навигации

4.1 Калибровка датчиков

Сырые показания МЭМС-датчиков содержат постоянное смещение нуля (bias), обусловленное технологическими особенностями изготовления кристалла [?]. Смещения определялись экспериментально по данным CSV-файла в состоянии покоя. Перевод в физические единицы выполняется линейным масштабированием:

$$a_i = (\tilde{a}_i - b_{a,i}) \cdot S_a, \quad \omega_i = (\tilde{\omega}_i - b_{\omega,i}) \cdot S_\omega, \quad (1)$$

где $\tilde{a}_i, \tilde{\omega}_i$ — сырые целочисленные отсчёты; $b_{a,i}, b_{\omega,i}$ — смещения нуля; $S_a = 9,81/3050$ [м/с²/отсчёт], $S_\omega = 0,001$ [рад/с/отсчёт].

Экспериментально определённые смещения, вынесенные в пространство имён `Calib` в файле `imu_shared.h`:

$$b_a = (31, 24, 0)^\top, \quad b_\omega = (-7, 1,5, 3)^\top. \quad (2)$$

4.2 Представление ориентации кватернионами

Ориентация тела в пространстве описывается единичным кватернионом [?]:

$$q = q_w + q_x i + q_y j + q_z k, \quad \|q\| = 1. \quad (3)$$

Кватернионы предпочтительнее матриц вращения и углов Эйлера: они не страдают от шарнирного замка (*gimbal lock*), компактны (4 параметра против 9) и численно стабильны при нормализации [?].

4.3 Кватернионное интегрирование гироскопа

Кинематическое уравнение связывает производную кватерниона с угловой скоростью $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^\top$ в системе координат тела [?]:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{pmatrix} 0 \\ \boldsymbol{\omega} \end{pmatrix}. \quad (4)$$

Для численного интегрирования на шаге Δt применяется точная формула через ось-угол [?]:

$$\theta = \|\boldsymbol{\omega}\| \cdot \Delta t, \quad \Delta \mathbf{q} = \left(\cos \frac{\theta}{2}, \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|} \sin \frac{\theta}{2} \right). \quad (5)$$

Обновление кватерниона с нормализацией:

$$\mathbf{q} \leftarrow \frac{\mathbf{q} \otimes \Delta \mathbf{q}}{\|\mathbf{q} \otimes \Delta \mathbf{q}\|}. \quad (6)$$

4.4 Матрица поворота из кватерниона

Для перевода вектора ускорения из системы координат тела в мировую используется матрица вращения $\mathbf{R} \in SO(3)$ [?]:

$$\mathbf{R} = \begin{pmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y-q_wq_z) & 2(q_xq_z+q_wq_y) \\ 2(q_xq_y+q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z-q_wq_x) \\ 2(q_xq_z-q_wq_y) & 2(q_yq_z+q_wq_x) & 1-2(q_x^2+q_y^2) \end{pmatrix}. \quad (7)$$

4.5 Углы Эйлера (конвенция ZYX)

Из кватерниона вычисляются углы крена ϕ , тангажа ϑ и рыскания ψ [?]:

$$\phi = \text{atan2}(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)), \quad (8)$$

$$\vartheta = \arcsin(2(q_w q_y - q_z q_x)), \quad (9)$$

$$\psi = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)). \quad (10)$$

4.6 Двойное интегрирование (скорость и позиция)

Ускорение переводится из системы тела в мировую, из него вычитается гравитация [?]:

$$a_{\text{world}} = R a_{\text{body}} - g, \quad g = (0, 0, -9,81)^\top \text{ [м/с}^2\text{]}. \quad (11)$$

Интегрирование методом Эйлера первого порядка:

$$v(t+\Delta t) = v(t) + a_{\text{world}}(t) \cdot \Delta t, \quad (12)$$

$$p(t+\Delta t) = p(t) + v(t) \cdot \Delta t. \quad (13)$$

4.7 Метод ZUPT

ZUPT (*Zero Velocity Update*) — классический способ частичной коррекции INS в моменты покоя [?]. Если норма угловой скорости опускается ниже порогового значения $\omega_{\text{th}} = 0,015$ рад/с, скорость мягко гасится с коэффициентом $\alpha = 0,85$:

$$\text{если } \|\omega\| < \omega_{\text{th}} : \quad v \leftarrow v \cdot \alpha. \quad (14)$$

Дополнительно, если компонента мирового ускорения по модулю меньше $0,3 \text{ м/с}^2$, она обнуляется полностью, что устраняет интегрирование шумового

сигнала акселерометра в статике.

5 Реализация навигационного алгоритма

5.1 Функция `integrate_imu()`

Весь навигационный алгоритм сосредоточен в функции `integrate_imu()` файла `imu_visualizer.cpp`. Она вызывается из потока визуализации для каждого пакета, извлечённого из очереди. Шаг интегрирования Δt вычисляется как разность временны́х меток двух последовательных пакетов.

Блок 1. Калибровка сырых отсчётов по формуле (??):

```
1 double ax = (raw.xacc - Calib::AX_BIAS) * Calib::ACCEL_SCALE;
2 double ay = (raw.yacc - Calib::AY_BIAS) * Calib::ACCEL_SCALE;
3 double az = (raw.zacc - Calib::AZ_BIAS) * Calib::ACCEL_SCALE;
4 double gx = (raw.xgyro - Calib::GX_BIAS) * Calib::GYRO_SCALE;
5 double gy = (raw.ygyro - Calib::GY_BIAS) * Calib::GYRO_SCALE;
6 double gz = (raw.zgyro - Calib::GZ_BIAS) * Calib::GYRO_SCALE;
```

Листинг 4: Калибровка сырых показаний

Блок 2. Кватернионное интегрирование через ось-угол по формулам (??)–(??):

```
1 double theta = std::sqrt(gx*gx + gy*gy + gz*gz) * dt;
2 if (theta > 1e-10) {
3     double norm_w = std::sqrt(gx*gx + gy*gy + gz*gz);
4     double s = std::sin(theta * 0.5) / norm_w;
5     double c = std::cos(theta * 0.5);
6     double dqw = c,    dqx = gx*s, dqy = gy*s, dqz = gz*s;
7
8     // q_new = q (x) dq
9     double qw = n.qw*dqw - n.qx*dqx - n.qy*dqy - n.qz*dqz;
10    double qx = n.qw*dqx + n.qx*dqw + n.qy*dqz - n.qz*dqy;
11    double qy = n.qw*dqy - n.qx*dqz + n.qy*dqw + n.qz*dqx;
12    double qz = n.qw*dqz + n.qx*dqy - n.qy*dqx + n.qz*dqw;
13
```

```

14     double norm = std::sqrt(qw*qw + qx*qx + qy*qy + qz*qz);
15     n.qw=qw/norm; n.qx=qx/norm; n.qy=qy/norm; n.qz=qz/norm;
16 }

```

Листинг 5: Кватернионное интегрирование гироскопа

Блок 3. Перевод ускорения в мировую систему координат, вычитание гравитации и применение ZUPT:

```

1 // Матрица поворота строки( r00..r22) из кватерниона
2 double ax_w = r00*ax + r01*ay + r02*az;
3 double ay_w = r10*ax + r11*ay + r12*az;
4 double az_w = r20*ax + r21*ay + r22*az;
5 az_w -= (-9.81);          // вычитаем гравитацию
6
7 // ZUPT: гироскоп в покое -> мягко гасим скорость
8 double gyro_norm = std::sqrt(gx*gx + gy*gy + gz*gz);
9 if (gyro_norm < Calib::ZUPT_GYRO_THRESH) {
10     n.vx *= 0.85; n.vy *= 0.85; n.vz *= 0.85;
11     if (std::abs(ax_w) < 0.3) ax_w = 0;
12     if (std::abs(ay_w) < 0.3) ay_w = 0;
13     if (std::abs(az_w) < 0.3) az_w = 0;
14 }
15 // Двойное интегрирование
16 n.vx += ax_w*dt; n.vy += ay_w*dt; n.vz += az_w*dt;
17 n.px += n.vx*dt; n.py += n.vy*dt; n.pz += n.vz*dt;

```

Листинг 6: Мировое ускорение, компенсация гравитации и ZUPT

5.2 Вычисление углов Эйлера

Завершает функцию вычисление углов Эйлера по формулам (??)–(??). Аргумент `asin` зажимается в интервал $[-1, 1]$ для защиты от накопленной численной погрешности:

```

1 const double R2D = 180.0 / M_PI;
2 n.roll = std::atan2(2*(qw*qx + qy*qz),

```



```

3             1 - 2*(qx*qx + qy*qy)) * R2D;
4 double sp = std::max(-1.0, std::min(1.0, 2*(qw*qy - qz*qx)));
5 n.pitch = std::asin(sp) * R2D;
6 n.yaw   = std::atan2(2*(qw*qz + qx*qy),
7             1 - 2*(qy*qy + qz*qz)) * R2D;

```

Листинг 7: Углы Эйлера из кватерниона (ZYX)

6 Реализация визуализации

6.1 Шейдеры OpenGL 3.3

Визуализатор использует две пары GLSL-шейдеров. Первая (VERT3D/FRAG3D) применяется для рендеринга 3D-модели полётного контроллера с диффузным освещением по модели Ламберта:

```
1 // VERT3D
2 #version 330 core
3 layout(location=0) in vec3 aPos;
4 layout(location=1) in vec3 aColor;
5 layout(location=2) in vec3 aNormal;
6 out vec3 vColor; out vec3 vNormal; out vec3 vPos;
7 uniform mat4 uMVP; uniform mat4 uModel;
8 void main(){
9     vPos    = vec3(uModel * vec4(aPos, 1));
10    vNormal = mat3(transpose(inverse(uModel))) * aNormal;
11    vColor  = aColor;
12    gl_Position = uMVP * vec4(aPos, 1);
13 }
14
15 // FRAG3D
16 #version 330 core
17 in vec3 vColor; in vec3 vNormal; in vec3 vPos;
18 out vec4 outColor;
19 uniform vec3 uLight; uniform float uAlpha;
20 void main(){
21     float d = max(dot(normalize(vNormal), normalize(uLight)), 0.0);
22     outColor = vec4(vColor * (0.35 + 0.65*d), uAlpha);
23 }
```

Листинг 8: Вершинный и фрагментный шейдеры 3D-модели

Вторая пара (VERT_FLAT/FRAG_FLAT) используется для отрисовки всех линейных примитивов без освещения: координатной сетки, мировых осей,

траектории и HUD-графиков.

6.2 Процедурная модель полётного контроллера

3D-модель FC строится процедурно из параллелепипедов функцией `build_fc()`. Вспомогательная `push_box()` генерирует шесть граней куба с различными коэффициентами яркости для имитации объёма; `push_arrow()` добавляет три цветных стрелки осей (X — красная, Y — зелёная, Z — синяя):

```
1 static std::vector<V3> build_fc() {
2     std::vector<V3> v;
3     // Корпус -- тёмносерый- прямоугольник
4     push_box(v, 0, 0, 0, 0.8f,0.8f,0.14f, 0.13f,0.14f,0.19f)
5     // Белая полоска нос"" направление( Y+)
6     push_box(v, 0, 0.37f, 0.08f, 0.7f,0.06f,0.04f, 0.95f,0.95f,0.95f)
7     // Боковые рёбра
8     push_box(v, 0.37f, 0, 0.08f, 0.04f,0.8f,0.04f, 0.22f,0.22f,0.28f)
9     push_box(v, -0.37f, 0, 0.08f, 0.04f,0.8f,0.04f, 0.22f,0.22f,0.28f)
10    // Стрелки осей
11    push_arrow(v, 0,0,0.1f, 1,0,0, 0.7f, 0.95f,0.10f,0.10f); // X
12    push_arrow(v, 0,0,0.1f, 0,1,0, 0.7f, 0.10f,0.92f,0.10f); // Y
13    push_arrow(v, 0,0,0.07f, 0,0,1, 0.7f, 0.15f,0.42f,0.98f); // Z
14    return v;
15 }
```

Листинг 9: Процедурная сборка модели FC

6.3 Главный цикл рендера

Функция `run_visualizer()` инициализирует контекст GLFW/GLEW, компилирует шейдеры, загружает геометрию в видеопамять и запускает главный цикл. Схема каждого кадра:

1. Обработать события ввода (мышь, клавиши).

2. Вычитать из `SharedBuffer` все накопившиеся пакеты, для каждого вызвать `integrate_imu()`, добавить точку траектории и обновить кольцевые буферы.
3. Обновить матрицы орбитальной камеры, следящей за FC.
4. Отрисовать **3D-сцену** (верхние 72% экрана): сетку, мировые оси, траекторию с цветовым затуханием, модель FC.
5. Отрисовать **HUD-панель** (нижние 30% экрана): четыре графика временных рядов.

Матрица модели FC формируется из текущей позиции и кватерниона ориентации с помощью GLM:

```
1 glm::mat4 fc_pos = glm::translate(glm::mat4(1),
2   glm::vec3((float)nav.px, (float)nav.py, (float)nav.pz));
3 glm::quat  q((float)nav.qw, (float)nav.qx,
4   (float)nav.qy, (float)nav.qz);
5 glm::mat4 fc_model = fc_pos * glm::mat4_cast(q);
6 glm::mat4 fc_mvp   = vp * fc_model;
```

Листинг 10: Матрица модели из позиции и кватерниона

Траектория отрисовывается как набор отрезков с цветовым градиентом: старые точки темнее, новые — ярче:

```
1 float fade_step = 1.0f / buf.trail.size();
2 for (int i = 0; i < (int)buf.trail.size()-1; i++) {
3   float alpha = (float)i * fade_step;
4   float cr = 0.1f + 0.7f * alpha; // красный: 0.1 -> 0.8
5   float cg = 0.5f + 0.4f * alpha; // зелёный: 0.5 -> 0.9
6   float cb = 0.9f;                // синий: константа
7   // ... добавить отрезок trail[i] -> trail[i+1]
8 }
```

Листинг 11: Траектория с цветовым затуханием

6.4 HUD-панель графиков

Нижняя панель содержит четыре независимых графика на основе скользящего буфера RingBuf (300 последних значений). Каждый график отображает три сигнала (по осям X, Y, Z) красным, зелёным и синим цветами:

Таблица 4 – Содержимое HUD-панелей

Панель	Диапазон	Сигналы
Акселерометр	$\pm 15 \text{ м/с}^2$	a_x (кр.), a_y (зел.), a_z (синий)
Гироскоп	$\pm 0,15 \text{ рад/с}$	$\omega_x, \omega_y, \omega_z$
Углы Эйлера	$\pm 180^\circ$	крен, тангаж, рыскание
Скорость	$\pm 2 \text{ м/с}$	v_x, v_y, v_z

7 Инструкция по установке и запуску

7.1 Установка зависимостей

```
1 sudo apt update
2 sudo apt install build-essential cmake git \
3     libglfw3-dev libglew-dev libglm-dev libgl1-mesa-dev
```

Листинг 12: Установка системных зависимостей (Ubuntu)

Библиотека MAVLink является заголовочной и клонируется напрямую в каталог проекта:

```
1 cd pixhawk_imu_visualizer
2 git clone https://github.com/mavlink/c_library_v2.git
```

Листинг 13: Клонирование MAVLink

7.2 Сборка проекта

```
1 mkdir build && cd build
2 cmake ..
3 make -j$(nproc)
```

Листинг 14: Сборка через CMake

При успешной сборке в каталоге `build/` появляется исполняемый файл `pixhawk_imu_reader`.

7.3 Настройка прав и запуск

```
1 # Добавить пользователя в группу dialout необходима( перезагрузка)
2 sudo usermod -a -G dialout $USER
3 # Или временно:
4 sudo chmod 666 /dev/ttyACM0
5
6 cd build
```

```
7 | ./pixhawk_imu_reader
```

Листинг 15: Права на порт и запуск

Окно визуализации открывается немедленно. Для завершения — Ctrl+C в терминале.

7.4 Управление в окне визуализации

Таблица 5 – Управление в окне визуализации

Действие	Эффект
ЛКМ + перетаскивание	Вращение камеры вокруг объекта
Колесо мыши	Приближение / отдаление (0,5 — 30 м)
Клавиша R	Сброс позиции и траектории в (0, 0, 0)
Ctrl+C	Завершение программы

8 Результаты работы

8.1 Внешний вид программы

На рисунке ?? представлен общий вид окна визуализатора в реальном времени. Верхняя область содержит 3D-сцену с моделью полётного контроллера на фоне координатной сетки и мировых осей; нижняя — четыре графика датчиков.

Рис. 1 – Главное окно визуализатора: 3D-сцена и HUD-панель

На рисунке ?? показано состояние визуализатора при вращении полётного контроллера: 3D-модель следует за реальной ориентацией устройства, а кривые гироскопа на HUD-панели фиксируют угловые скорости по каждой из осей.

Рис. 2 – Визуализатор при вращении устройства: реакция HUD-графиков гироскопа

На рисунке ?? приведён укрупнённый вид HUD-панели. Нулевые линии в каждом графике выделены серым. Видно, что в статике (покой платформы) кривые гироскопа сходятся к нулю — это подтверждает корректность подобранных смещений (??).

Рис. 3 – Укрупнённый вид HUD-панели: акселерометр, гироскоп, углы Эйлера, скорость

8.2 Результаты оценки ориентации

В ходе тестирования подтверждена корректная работа алгоритма кватернионного интегрирования. При вращении полётного контроллера по всем трём осям 3D-модель точно воспроизводила движения устройства.

Углы Эйлера, вычисленные по формулам (??)–(??), соответствовали физическим поворотам.

8.3 Результаты оценки позиции

Алгоритм двойного интегрирования ускорения показал неудовлетворительные результаты при оценке абсолютных координат. Ключевая проблема — **дрейф позиции**: уже через 5–10 с накопленная ошибка достигала нескольких метров. Механизм ZUPT снизил скорость нарастания дрейфа в состоянии покоя, однако не устранил его полностью из-за ненулевого шума акселерометра порядка 10^{-3} – 10^{-2} м/с².

8.4 Анализ причин дрейфа

Пусть истинное ускорение $a(t)$ и измеренное $\hat{a}(t)$ связаны:

$$\hat{a}(t) = a(t) + b + \eta(t), \quad (15)$$

где b — остаточное смещение после калибровки, $\eta(t)$ — случайный шум. Ошибка позиции после двойного интегрирования за время T :

$$\delta p(T) = \frac{1}{2} b T^2 + \int_0^T (T - t) \eta(t) dt. \quad (16)$$

Первое слагаемое растёт *квадратично* по времени даже при малом остаточном смещении [?]. За 10 с это даёт ошибку порядка нескольких метров [?].

8.5 Возможные направления развития

Полноценная инерциальная навигация без внешних корректирующих источников принципиально невозможна [?]. Среди возможных улучшений:

1. **Фильтр Калмана с GPS-коррекцией** [?] — классический подход для авиационных INS.
2. **Фильтр Маджвика** [?] — градиентный алгоритм слияния данных гироскопа, акселерометра и магнитометра.
3. **Нелинейный дополняющий фильтр на $SO(3)$** [?].
4. **ZUPT с фильтром Калмана** [?] — расширенное применение метода нулевой скорости.

Заключение

В ходе выполнения курсовой работы разработана система визуализации инерциальной навигации на основе полётного контроллера Pixhawk. Реализованы следующие компоненты:

- автономный модуль первичного считывания RAW_IMU с записью данных в CSV для анализа и подбора калибровочных коэффициентов;
- многопоточная архитектура C++14 с потокобезопасным SharedBuffer на основе `std::mutex` и `std::atomic`;
- алгоритм калибровки сырых показаний с экспериментально определёнными смещениями нуля (??);
- точное кватернионное интегрирование угловой скорости через ось-угол [?] с нормализацией на каждом шаге;
- вычисление матрицы вращения $R \in SO(3)$ и углов Эйлера из кватерниона;
- двойное интегрирование ускорения для оценки позиции [?];
- механизм ZUPT [?] для частичной компенсации дрейфа;
- OpenGL 3.3-визуализация: процедурная 3D-модель FC, координатная сетка, мировые оси, траектория с цветовым затуханием, HUD-панель с четырьмя графиками [?].

Алгоритм оценки ориентации показал высокое качество: 3D-модель точно воспроизводила реальные повороты устройства. Алгоритм оценки позиции продемонстрировал принципиальное ограничение МЭМС-датчиков [?]: ошибка нарастает квадратично по времени (??), и

получение практически пригодных координат требует слияния данных IMU с корректирующими сенсорами в рамках фильтра Калмана [?].

Список использованных источников

Список литературы

- [1] Titterton D., Weston J. *Strapdown Inertial Navigation Technology*. 2nd ed. — IET, 2004. — 558 p.
- [2] Woodman O.J. *An Introduction to Inertial Navigation*. Technical Report UCAM-CL-TR-696. — University of Cambridge, 2007. — 37 p. — URL: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf> (дата обр. 01.03.2024).
- [3] Kuipers J. B. *Quaternions and Rotation Sequences*. — Princeton University Press, 1999. — 398 p.
- [4] Mahony R., Hamel T., Pflimlin J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group // *IEEE Transactions on Automatic Control*. — 2008. — Vol. 53, No. 5. — P. 1203–1218. — DOI: 10.1109/TAC.2008.923738.
- [5] Madgwick S.O.H., Harrison A.J.L., Vaidyanathan R. Estimation of IMU and MARG orientation using a gradient descent algorithm // *IEEE International Conference on Rehabilitation Robotics (ICORR)*. — 2011. — P. 1–7. — DOI: 10.1109/ICORR.2011.5975346.
- [6] Groves P.D. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. 2nd ed. — Artech House, 2013. — 776 p.
- [7] Skog I., Handel P. In-Car Positioning and Navigation Technologies — A Survey // *IEEE Transactions on Intelligent Transportation Systems*. — 2009. — Vol. 10, No. 1. — P. 4–21. — DOI: 10.1109/TITS.2008.2011712.
- [8] Meier L. et al. MAVLink: A Lightweight Message Marshalling Library

for the DroneCode Ecosystem. — URL: <https://github.com/mavlink/mavlink> (дата обр. 01.03.2024).

[9] MAVLink Developer Guide: Common Messages. — URL: <https://mavlink.io/en/messages/common.html> (дата обр. 01.03.2024).

[10] PX4 Autopilot User Guide. — URL: <https://docs.px4.io/main/en/> (дата обр. 05.03.2024).

[11] GLFW — An OpenGL library. Documentation. — URL: <https://www.glfw.org/docs/latest/> (дата обр. 05.03.2024).

[12] The Open Group. The Single UNIX Specification, Version 4. *termios.h* — *define values for termios*. — URL: <https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/termios.h.html> (дата обр. 01.03.2024).